

Feature: The booking

From build-plan: feature 5d

Branch: feature/05d-the-booking

Status: verified. Whole feature seen and agreed by Frank 2026-10-02; steps 5d.1 to 5d.5 built, tested and reviewed step by step; no P0 or P1 was ever open; the final review's F-92 and F-93 fixed last on 2026-10-02. The checkpoint for the final review.

Goal

A customer picks one of 5c's free times and books it: the time is checked again, the person (and the room, when the service needs one) is held, and the contact, the lead in the first stage, the booking with the customer's address and the timeline entry are written together, all or nothing. Then the event goes into the booked person's Google calendar. The public route the widget (feature 9) will call does this for strangers; the same function makes a booking the owner enters (a phone estimate, a walk-in), whose route arrives with its screen.

The last of feature 5's four parts. Nothing shows on screen yet.

In scope

- The `lead` and `booking` tables, and `commitment.bookingId` pointing at its booking.
- Writing a booking as one unit: `holdTime`, `releaseTime`, `findOrCreateContact` and `recordActivity` can join a caller's transaction, and a refused hold leaves that transaction usable (F-52).
- One room rule, `isRoomFree`, shared by the free times and the booking (F-64).
- `bookTime`: check again, choose who gets "any available", hold, write everything, then the Google event.
- Writing the event into the booked person's Google calendar, and one helper that hands out a fresh access token to both the busy-time read and the event write.
- `POST /public/:slug/bookings`, and the times route answering "try again shortly" when nobody's calendar could be read (F-74).

Decisions

Made in this spec as the simplest safe option (reversible, recorded; each gets a line in the build log's Decisions drawer with why the other option lost):

1. **An owner-made booking is a function now, a route with its screen** (overview: CRM routes arrive with their screens). `bookTime` takes `source: "manual"` and the owner's login; that login must be a member

of the business, checked inside `bookTime` , never trusted. 2. **A booking's `startsAt` and `endsAt` are the appointment itself.** The buffers live only in its commitments, which span `startsAt - bufferBefore` to `endsAt + bufferAfter` , as 5c reads them. 3. **The cancel token waits for feature 7.** Nothing reads it before then, and a token is only useful in the email that carries it (feature 6). The plan's `cancelToken` column arrives with them. 4. **Every booking makes a new lead**, in the business's first stage, even for a known contact: a returning customer is a new job. 5. **A refused hold tries the next choice, never a stale one.** With "any available", the free people are tried in decision 2's order (fewest bookings that day, then name, then id), each with the free rooms by name, inside the same transaction, until one holds. A picked person who was taken meanwhile answers `time_taken` . 6. **Google is written after the booking is saved, and a failure keeps the booking.** A customer's booking is never lost because Google hiccuped: the booking stays without `calendarEventId` , one warning line is logged, and writing the event again is feature 8's background job. The event never has attendees, so Google never emails the customer from a worker's account (feature 6: customers only hear from the business). The customer's answer never waits for Google (Frank, 2026-10-02, review F-93, option A): the write starts once the booking is saved and finishes on its own, so a slow Google cannot keep the Book button spinning. Rejected: waiting (up to 20 seconds when Google is slow, and a browser that gives up shows a failure for a booking that exists). If the server restarts in that moment, the event waits for feature 8's retry. 7. **The same form sent twice gets one booking** (Frank, 2026-10-02, after review F-75). The widget makes a one-time `requestKey` when the booking form opens and sends it with Book; `booking` stores it, unique per business, so the database lets only one booking in for a key, even when two copies arrive at the same instant (a double tap, or a press repeated after a lost answer). A request whose key is already booked answers that booking, when it asks for the same service and start (and the same person, if one was picked); a key already used for a different booking is refused, `request_key_used` (review F-81). A different key is a new booking: a parent booking two children at 9:00 is two forms, two keys, two bookings. Owner-made bookings carry no key. The widget also locks its Book button after one press (feature 9); the key covers what a button cannot, since the server never trusts the front end. Rejected: refusing a second booking for the same email, service and start, which would refuse the second child. 8. **The timeline entry links the lead and the booking in its payload** (`{ leadId, bookingId, bookingLinkId, startsAt }`), not new columns; feature 11 reads them. 9. **"Any available" with every calendar unreadable is "try again shortly"** (F-74): the times route answers 503 `unavailable` when every offered person was left out because their calendar could not be read, instead of an empty week that reads as fully booked. One readable person is enough for 200. 10. **A public booking request is at most 16 KB**, checked with Hono's own `bodyLimit` before it is parsed (no new package). 11. **The owner books any time someone is not busy** (Frank, 2026-10-02, open question 2, option B; already the plan's word in feature 12b: bookable hours only limit customers). An owner-made booking ignores bookable hours, notice, the horizon and standby, but never goes over a booking, time off or the person's own Google busy time: Google wins (a dinner in Pedro's business calendar blocks his bookable evening, for customers and for the owner). The room, when needed, must be free by the same room rule, standby ignored. People often book themselves this way rather than change their settings. 12. **"Time taken" says it happened while they were booking** (Frank, 2026-10-02). The time can stop being free between seeing it and pressing Book: another customer booked it, the owner booked or gave time off, the person added something to their Google, or the page

stayed open until the notice passed. The server cannot always tell which, so the message is true for all of them and says it is not the customer's fault: "Sorry, that time was taken while you were booking. Please pick another one." The widget (feature 9) shows it with the times still free. Rejected: "someone booked that time a moment before you", true only for the first. 13. **The owner's own bookings start no earlier than today** (Frank, 2026-10-02, review F-82, option B). Any time from the start of today in the business's zone, so a walk-in already under way can be entered; an earlier day (or a mistyped year) answers `in_the_past`. Rejected: any start (a typo goes straight through) and only from now on (a walk-in that began ten minutes ago would be refused). Customers are never affected. 14. **The Google event carries what the worker needs on site** (Frank, 2026-10-02, open question 3, option A). Title: the service and the customer's name ("Interior estimate: Jane Doe"); location: the address; description: "Phone: ...", "Email: ..." (each only when given) and what they wrote. Never attendees. Rejected: the service and first name only, which leaves a painter on site without the address or phone. 15. **A returning customer's phone travels with the booking** (Frank, 2026-10-02, review F-85, option A). The phone given with a request is kept on its lead (`lead.phone` , migration 0015) and is the one its event shows; the saved contact gets a phone only when it had none, so typing someone's email on the public form never changes their saved details. Rejected: the newest details always win (anyone could change a contact by typing their email) and keeping only the first (the new phone was lost). 16. **An online booking gives a phone or an email, at least one** (Frank, 2026-10-02, open question 1). The form asks for both, in two fields, with the hint "At least one is required"; both is best. There must be a way to reach the customer; whichever is given is what feature 6 uses (email only: an email; phone only: a text). It never changes the free times, which come from the business's own calendars. A phone-only customer has no email to be matched by, so each such booking makes its own contact; Frank accepted the duplicates for now, matching by phone is a note for the CRM features. Which of the two a business requires can become its own setting later (feature 12). Rejected: email always required (a business that works by phone loses those bookings). An owner-made booking keeps both optional, as `contact` allows.

Open questions

Each blocks only the step named; Frank answers it when that step's plan is gone through, before it is built.

1. Answered 2026-10-02: decision 16. 2. Answered 2026-10-02: decision 11. 3. Answered 2026-10-02: decision 14.

Out of scope

- Confirmation emails and texts (6); cancel and reschedule, and the cancel token (7); retrying a Google event that failed (8).
- The widget (9) and the owner's booking screen (11 or 12b); the owner-made booking's route arrives with that screen.
- Rate limiting public routes (noted for 9, the first public traffic; F-62): per visitor, and per contact, so the same email or phone booking a burst in a short while is refused (Frank, 2026-10-02, keeping decision

7).

- Taking deposits or payment.

Build loop

Steps are built one at a time on `feature/05d-the-booking`. Each step's plan gets Frank's yes just before it is built. After that yes nothing stops until the review: build, tests, tick the box, the build log entry, commit with the step number and push to the feature branch, `/audit` scoped to the step, then the independent review (`workflow.stepReview: "every"`, `workflow.checkpointCommits: "enabled"`). Findings are talked through after the review; P0/P1 are fixed before the next step. `/complete` makes the merge commit, on Frank's yes.

Build steps

- ☒ **5d.1 The lead and booking tables.**
 - `packages/shared/db/crm-tables/lead-table.ts`: `id`, `organizationId`, `contactId` (same business, cascade), `stageId` (same business, no action: a stage with leads cannot be deleted), `source` (`widget`, `hosted`, `manual`, check `lead_source_check`), `details` (text, nullable: what the customer wrote), timestamps; unique (`organizationId`, `id`); index (`organizationId`, `stageId`).
 - `packages/shared/db/booking-tables/booking-table.ts`: `id`, `organizationId`, `leadId`, `bookingLinkId`, `personId`, `placeId` (nullable), each a same-business foreign key with no action; `startsAt`, `endsAt` (the appointment, `booking_time_order_check`); `status` (`confirmed` | `cancelled`, default `confirmed`, `booking_status_check`); `location` (text, not empty after trimming, `booking_location_check`); `calendarEventId` (nullable); timestamps; unique (`organizationId`, `id`); index (`organizationId`, `startsAt`).
 - `commitment.bookingId` gains `commitment_booking_fk` to (`organizationId`, `id`) on `booking`, no action.
 - Migration `0013_lead_and_booking.sql`, generated from `packages/shared`. If `drizzle-kit` writes a foreign key before the unique it needs (the 0010 trap, `42830`), the statement is moved by hand and a later `db:generate` must find no change.
 - **Done when** `db:migrate` builds a fresh `scheduleads_dev`, `db:seed` runs twice, `db:generate` finds no change, and database tests prove: a lead in another business's stage or for another business's contact is refused; an unknown source is refused; a booking with an empty location, an end before its start, an unknown status, or another business's lead, service, person or place is refused; a commitment pointing at another business's booking is refused; the backend and frontend builds pass.
 - Review fix (F-75, decision 7): `booking.requestKey` (text, nullable) and `booking_request_key_unique` on (`organizationId`, `requestKey`) where it is not null, in

migration `0014_booking_request_key.sql` (0013 had already run on the dev database, which holds a real Google connection). **Done when** a test proves a second booking with the same key in the same business is refused, the same key in another business and two bookings without a key are both allowed.

- **✓ 5d.2 Writing as one: transactions and the room rule.**

- `holdTime`, `releaseTime`, `findOrCreateContact` and `recordActivity` take an optional last argument, the executor (`db` or a transaction), default `db`. `holdTime` runs each attempt in a nested transaction (a savepoint inside the caller's), so `{ held: false }` and a deadlock retry leave the caller's transaction usable (F-52).
- `backend/lib/scheduling/is-room-free.ts: isRoomFree(room, date, spanStart, spanEnd)`, the room rule pulled out of `applyFreeTimesRules` (not on standby that date, no busy block over the span, half-open). It takes any room with its taken time and standby dates; a caller keeps its own ids (5d.3 filters its rooms with it). The span itself comes from one helper, `appointmentSpan` (the buffer before, the appointment, the buffer after; review F-76). `applyFreeTimesRules` uses both, behaviour unchanged (F-64).
- **Done when** saved tests prove: inside a transaction, a taken hold answers `{ held: false }` and the transaction still commits its other rows; a rolled-back transaction leaves no commitment, contact or activity; each of the four functions writes nothing outside the transaction it was given; `isRoomFree` covers standby, an overlapping block, a touching block and the buffers; every existing test passes unchanged.

- **✓ 5d.3 Booking a time.**

- `backend/lib/booking/book-time.ts: bookTime({ organizationId, bookingLinkId, personId, startsAt, customer, location, details, source, actorUserId, now })`, `personId` null for "any available". Answers `{ booked: true, booking, alreadyBooked }` or `{ booked: false, reason }` with `reason` one of `not_found`, `time_taken`, `unavailable`.
- Check again with 5c's own reads: the start must be one `findFreeTimes` offers for that person on that date. Owner-made (decision 11): instead, the span (`appointmentSpan`) must touch none of the person's bookings, time off or Google busy time, and a needed room must pass `isRoomFree` with standby ignored. With "any available", each offered person is checked; their order comes from `chooseAnyAvailable` with `countBookingsThatDay`, the rooms from `isRoomFree`. A picked person whose calendar cannot be read answers `unavailable`.
- Decision 7 runs first, before the check: a request whose `requestKey` is already booked answers that booking (the customer's own first booking would otherwise make the time look taken). Inside the transaction the booking's insert carries the key; refused by `booking_request_key_unique` (two copies at the same instant), the transaction is rolled back and the booking that won is the answer. Then the check, then one transaction: `findOrCreateContact`, `findFirstPipelineStage`, the lead, the booking, `holdTime` (person, and room when needed, buffers inside, `bookingId` set; decision 5 on a refusal), and `recordActivity` (`booking_created`, decision 8, the owner's login as actor when owner-made). Nothing is left behind when it answers `time_taken`.

- **Done when** database tests on small clinics of their own prove: a picked practitioner's booking writes the contact, a lead in the first stage with its source, the booking with the address, the commitments (person and room, buffers inside, `bookingId` set) and the timeline entry; a known email reuses its contact; "any available" goes to the fewest bookings that day; a time that stopped being free answers `time_taken` and writes nothing; two bookings of the same person at the same instant give one booked and one `time_taken`, the loser leaving no contact, lead or booking; "any available" whose first choice is taken meanwhile goes to the next; the same form sent twice, one after the other and at the same instant, gets one booking and both answer it; two forms with different keys for the same email and time book twice; an unreadable picked calendar answers `unavailable`; a room taken only during the buffer after is not chosen, and the held span is `appointmentSpan`'s (F-76); another business's service or person answers `not_found`; an owner-made booking by someone outside the business is refused; an owner-made booking outside bookable hours, on a standby date or within the notice is booked, and one over the person's Google busy time, a booking or time off answers `time_taken` (decision 11).
- ✓ **5d.4 The event in the booked person's Google.**
 - The seam gains `createEvent(accessToken, event)`, answering the provider's event id; the Google plug inserts into the primary calendar with a timeout, no attendees, times in the business's zone.
 - `backend/lib/calendar/get-fresh-access-token.ts`: the refresh that `getBusyTimes` does today (the same race rules, the same `needs_reconnect` marking), pulled out so both use one copy.
 - `backend/lib/calendar/write-booking-event.ts`: writes the event for one booking and saves `calendarEventId`; no connection writes nothing. `bookTime` calls it after the transaction commits; any failure keeps the booking and logs one warning line with a safe reason (decision 6).
 - **Done when** saved tests with Google faked prove: the event's title, location, description (decision 14), start and end (the appointment, not the buffers) and zone; its id saved on the booking; no connection makes no call; a connection needing reconnection, a Google error and a timeout each keep the booking, leave `calendarEventId` empty and log one line; an expired token is refreshed through the shared helper; every `getBusyTimes` test passes unchanged.
- ✓ **5d.5 The public route.**
 - `POST /public/:slug/bookings` in a new `backend/routes/public-bookings-routes.ts`, mounted under `/public`. The body is checked by a shared Zod schema (`create-booking-validation-schema.ts` in `booking-links-validation-schemas/`, exported through `index.ts`) through Hono's own `validator("json")`, after `bodyLimit` (decision 10). The business comes only from the slug, as on the other public routes.
 - `publicCorsMiddleware` allows `POST` too, still without credentials.
 - `RefusalCodeType` gains `time_taken` and `request_key_used`.
 - The times route answers 503 `unavailable` per decision 9.

- **Done when** route tests on a clinic of their own (removed after, so a failed run leaves nothing in `clinic-dev` ; review F-91) prove: 201 with the booking's id, times, zone, service and person, and never the customer's name, email, phone or address, or `organizationId` ; 400 for a malformed body, a start that is not an instant, an empty address, and a customer with neither a phone nor an email (decision 16); 413 for a body over 16 KB; the one identical 404 for an unknown business, an inactive service and a person not offered; 409 `time_taken` with decision 12's message for a time no longer free; 409 `request_key_used` with "This booking form was already used. Please reload the page and book again." for a used key sent with a different booking; 503 `unavailable` for an unreadable picked calendar, and on the times route when every calendar is unreadable; a preflight allows `POST` without credentials; `npm run build --workspace=frontend` passes and the typed client sees the route (checked with a throwaway file, as in 5c.5).

Files / areas

- `packages/shared/db/crm-tables/lead-table.ts` , `packages/shared/db/booking-tables/booking-table.ts` , `packages/shared/db/scheduling-tables/commitment-table.ts` , `packages/shared/db/index.ts` , `packages/shared/migrations/0013_` and `0014_` .
- `packages/shared/zod-validation/booking-links-validation-schemas/` (the booking body schema, exported through `index.ts`).
- `backend/lib/scheduling/` : `hold-time.ts` , `release-time.ts` , `is-room-free.ts` , `apply-free-times-rules.ts` , `find-free-times.ts` .
- `backend/lib/crm/` : `find-or-create-contact.ts` , `record-activity.ts` .
- `backend/lib/booking/book-time.ts` (new area), tests beside it.
- `backend/lib/calendar/` : `calendar-provider.ts` , `google-calendar-provider.ts` , `get-fresh-access-token.ts` , `get-busy-times.ts` , `write-booking-event.ts` .
- `backend/lib/errors/refuse.ts` , `backend/middleware/public-middleware/public-cors-middleware.ts` , `backend/routes/public-bookings-routes.ts` , `backend/routes/public-booking-links-routes.ts` , `backend/app.ts` , and their tests.

Data / contracts

lead: `id` text, `organizationId` , `contactId` , `stageId` , `source` (`widget` | `hosted` | `manual`), `details` text null, `createdAt` , `updatedAt` . Online bookings use `widget` (the hosted page, `hosted` , is feature 24).

booking: `id` text, `organizationId` , `leadId` , `bookingLinkId` , `personId` , `placeId` null, `startsAt` , `endsAt` (timestampz, the appointment), `status` (`confirmed` | `cancelled`), `location` text not empty, `calendarEventId` text null, `requestKey` text null (unique per business when set, decision 7), `createdAt` , `updatedAt` . Its commitments find it through `commitment.bookingId` .

The booking body (`POST /public/:slug/bookings`):

```

{
  bookingLinkId: string; // the same rule as the id in the times route
  startsAt: string; // an ISO 8601 instant with Z or an offset, one 5c offered
  personId?: string; // left out = any available
  requestKey?: string; // one per form, made when it opens (decision 7); the same id r
  customer: {
    name: string; // contactValidationSchema's rule
    email?: string; // decision 16: an email or a phone, at least one
    phone?: string; // a form sends nothing rather than ""
  };
  location: string; // the customer's address, trimmed, 1 to 300 characters
  details?: string; // what they wrote, trimmed, at most 2000 characters
}

```

| Status | Body | |---|---| | 201 | { booking: { id, startsAt, endsAt, timezone, service: { id, name }, person: { id, name } } } (also when decision 7 answers an existing booking) | | 400 | bad_request with the first reason | | 404 | not_found , one identical answer for every "not here" | | 409 | time_taken : "Sorry, that time was taken while you were booking. Please pick another one." (decision 12) | | 409 | request_key_used : "This booking form was already used. Please reload the page and book again." (review F-81) | | 413 | the request is over 16 KB | | 503 | unavailable : "Times cannot be read right now. Try again shortly." |

bookTime answers { booked: true; booking: BookedType; alreadyBooked: boolean } or { booked: false; reason: "not_found" | "time_taken" | "unavailable" | "request_key_used" | "in_the_past" }, the last only for the owner (decision 13), whose route arrives with its screen. "Any available" with nobody free and a calendar that could not be read answers unavailable , never time_taken (review F-80). Nothing in its answer or its logs carries the customer's name, email, phone, address or words.

The Google event: no attendees; start and end the appointment with the business's timeZone ; title, location and description per decision 14; calendarEventId is Google's event id. One event per booking (review F-84): the id Google is given is the booking's without its dashes, so a second write (feature 8's retry, a lost answer) finds the event already there; a booking already written or cancelled is not written.

Testing

Backend Vitest with the local scheduleads_dev migrated and seeded. 5d.2's isRoomFree is pure. Every other test uses a small business of its own, removed after, or the seeded clinic for the route, and refuses any database that is not local and *_dev . Google is never called: fetch is stubbed in every file that could reach it; a person with no connection has no busy time, and the unreadable case is a connection marked needs_reconnect .

Each case on the feature's Simulate page ("What happens when you press Book?") is also a saved test under the same name, in the step that builds its rule.

Notes for the AI

- `commitment` refuses overlaps itself (`commitment_no_overlap` , migration 0009, compares the business too); never check-then-insert without it.
- 5c's reads are reused as they are: `findFreeTimes` , `findServiceResources` , `findStandbyDates` , `findCommitments` , `resolveBookableHours` , `chooseAnyAvailable` , `countBookingsThatDay` . Do not write a second copy of any rule.
- Inside a transaction a refused insert aborts it; only a savepoint keeps it usable (F-52). Drizzle's nested `tx.transaction()` is a savepoint.
- Public routes take the business only from the slug, give one identical 404 for every "not here", and never echo a contact's name, email, phone or address. Log lines carry ids and safe reasons only.
- The Google scopes already granted (`calendar.events.owned`) allow writing an event to the person's own calendar; no reconnect is needed.
- At deploy: confirm `btree_gist` on Railway and apply 0005 to 0015; the server needs tzdata 2026c or newer (carried from 5c).

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `db/crm-tables/lead-table.ts` and `db/booking-tables/booking-table.ts` : the lead (contact, stage, source `widget` | `hosted` | `manual` , the customer's words, and since 0015 the phone given with that request) and the booking (lead, service, person, room or none, the appointment's own start and end, `confirmed` | `cancelled` , a never-empty address, the Google event id, and the form's `requestKey` , unique per business when set). Every foreign key carries the business, so a row can only point inside its own business.
- `db/scheduling-tables/commitment-table.ts` : `bookingId` now references the booking of the same business, no action on delete, so a booking keeps its held rows, cancelled ones included.
- Migrations `0013_lead_and_booking.sql` , `0014_booking_request_key.sql` and `0015_lead_phone.sql` , generated and untouched.
- `zod-validation/booking-links-validation-schemas/create-booking-validation-schema.ts` : the booking form's one rule. The start must carry Z or an offset; the customer needs a name and an

email or a phone, at least one (decision 16); the address is 1 to 300 characters, the words at most 2000. The first broken rule is the message the customer sees.

backend: writing as one

- `database-executor-type.ts` : `DatabaseExecutorType` , the pool or a caller's open transaction. `holdTime` , `releaseTime` , `recordActivity` and `findOrCreateContact` take one as an optional last argument, so the booking writes them together or not at all.
- `lib/scheduling/hold-time.ts` : each try runs in a nested transaction (a savepoint), so a refused hold inside the booking's transaction is undone alone and the transaction stays usable (F-52's trap).
- `lib/scheduling/appointment-span.ts` , `overlaps-any.ts` , `is-room-free.ts` : one copy each of the span an appointment keeps busy (buffer before, appointment, buffer after), of the half-open overlap test, and of the room rule. The free times and the booking both call them, so they cannot disagree about which time or room is free.
- `lib/scheduling/order-any-available.ts` : the order "any available" tries, fewest bookings that day first, then name, then id, each person paired with the free rooms by name. It replaced the loops that called `chooseAnyAvailable` with a stand-in person (F-83); `chooseAnyAvailable` is now called only by its own test (F-94, carried).

backend: booking a time

- `lib/booking/book-time.ts` : the one path every booking takes. In order: the form's key (a form already booked answers that booking, a key used for a different booking is `request_key_used`); the service and who is offered; the check again (a customer only one of the free times offered, the owner any time nobody is busy, Google winning, decision 11; the owner not before today, decision 13); the free rooms; the try order; then one transaction writing the contact, the lead in the first stage, the booking, the held time (the first choice the database still lets in, `holdFirstFreeChoice`) and the timeline entry. Nothing is left behind when it answers `time_taken` .
- Before any refusal the form's key is read again (F-92): a copy of the form whose check ran after its first copy was saved would otherwise see that copy's own time as taken and tell a booked customer "time taken".
- "Any available" with nobody free answers `unavailable` when a calendar could not be read (one of them may be free), never `time_taken` (F-80).
- `lib/crm/find-or-create-contact.ts` : inserted first and read only when that changed nothing, so two bookings at once with one email make one contact. A known contact keeps the details first given; a phone is filled only when it had none (decision 15). A phone-only customer cannot be matched, so each of their bookings makes its own contact (decision 16, accepted for now).

backend: the Google event

- `lib/calendar/get-fresh-access-token.ts` : the key renewal the busy-time read did inside itself, moved out with its race rules, so reading busy times and writing events share one copy.

- `lib/calendar/write-booking-event.ts` : writes one saved booking into the booked person's primary calendar: the service and the customer's name as title, the address as location, phone, email and words as description, the appointment's own times in the business's zone, never attendees (decision 14). The event id given to Google is the booking's id without dashes, so a second write finds the event already there; a cancelled or already written booking is not written (F-84).
- `lib/booking/booking-event-writes.ts` : starts that write once the booking is saved and does not wait for it (decision 6, F-93, Frank's option A), keeping the one warning line on failure. The writes still running can be awaited, which the tests do; a shutdown hook could later. A write lost to a restart waits for feature 8's retry.

backend: the public route

- `routes/public-bookings-routes.ts` : `POST /public/:slug/bookings` . The size limit (16 KB, decision 10) runs before the body is read; Hono's own "malformed JSON" text answer is turned into the usual `bad_request` by the sub-app's `onError` , every other error still reaches a 500. The business comes only from the slug. Answers: 201 with the booking's id, times, zone, service and person, never the customer's details; 400; the one 404 for every "not here"; 409 `time_taken` (decision 12's words) or `request_key_used` ; 413; 503 `unavailable` .
- `lib/booking/find-bookable-organization-id.ts` and `lib/errors/not-bookable-here.ts` : moved out of the services file so both public route files use one lookup and one identical 404.
- `lib/scheduling/find-free-times.ts` : with "any available", no calendar read and at least one failed now throws, so the times route answers 503 instead of an empty week (decision 9, F-74).
- `middleware/public-middleware/public-cors-middleware.ts` : allows POST beside GET, still never credentials.

Tests

Every case on the feature's Simulate page is a saved test under the same name. Database tests build small businesses of their own and remove them; the route tests book into a clinic of their own (F-91). Google is faked in every file that could reach it. Two races are tested without timing: the first choice taken between the check and the hold (`book-time-taken-meanwhile.test.ts`) and a form resent while its first copy saves (`book-time-resent-while-saving.test.ts`). Final count: 358 backend and 90 shared tests.

Review history

Every step was reviewed by a fresh reviewer before the next began; no P0 or P1 was ever open. The first final review (target `aa237da`) passed and found F-92 and F-93; both were fixed, and the second final review below passed over the whole feature again.

Carried forward

- F-94 (P3): `chooseAnyAvailable` is used only by its test; the spec still names it.
- F-95 (P3): the no-wait test's cleanup hangs if that test fails.
- Rate limits per visitor and per contact are written into feature 9; each business's own booking questions into features 9 and 12.
- At deploy: `btree_gist` on Railway, migrations 0005 to 0015, time zone data 2026c or newer.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47, F-58, F-62, F-94, F-95 stay in the live ledger.

5d/F-52 [P3] closed - holdTime and releaseTime cannot join a caller's transaction, which 5d and feature 7 need

File: backend/lib/scheduling/hold-time.ts:33 **Found:** 2026-10-01 by /audit independent (scope: step 5a.2, 1def0b9..d5175ae; lens: quality) **Why it matters:** The declared deviation holds for this step: one `INSERT ... VALUES` is atomic in Postgres, and its foreign key checks run inside the same statement, so a refused row takes the whole hold with it (the cross-business test proves `mine.ana` gets no row). But both functions always use the global `db`. 5d writes the booking, its commitments and the timeline entry together, and feature 7's reschedule must release the old time and hold the new one together; neither can be all-or-nothing through these functions as written. Inside a transaction, a `23P01` also aborts the whole transaction, so answering `{ held: false }` there needs a savepoint. **Suggested fix:** Nothing to change in 5a. Decide in 5d's spec: let both take an optional executor (`db` or a transaction) and hold inside a nested transaction (savepoint) so "taken" leaves the caller's transaction usable. **Resolution:** Closed 2026-10-02 by independent review of step 5d.2 (7846a1d..230181c): holdTime, releaseTime, findOrCreateContact and recordActivity take an optional executor, and holdTime runs each try in executor.transaction (postgres-js savepoint, rethrowing the original DrizzleQueryError so cause.code is still read); writing-as-one-transaction.test.ts proves a refused hold answers `{ held: false }` with the caller's transaction committing its other rows, and a release plus a hold in one transaction invisible outside it until commit. A scratch probe (40 pairs of opposite-order holds, each inside its own db.transaction) saw 15 deadlocks in pg_stat_database, every pair still one held and one taken, every outer transaction still usable; the saved test for that path is F-77.

5d/F-64 [P3] closed - chooseAnyAvailable needs "only the free rooms", but no code says which rooms are free, so 5d would rebuild the room rule

File: backend/lib/scheduling/apply-free-times-rules.ts:87 **Found:** 2026-10-02 by independent review of step 5c.3 (scope: 4d6d1ce..4858600; lenses: all) **Why it matters:** The contract trusts the caller to pass only free people and only free rooms. Free people can be found by running `applyFreeTimesRules` for one person and one date and checking the start is in the answer. Free rooms cannot: the room rule (not on standby that date, and no busy block over the appointment plus both buffers, half-open) lives inline at

lines 87 to 95, and `FreeTimesRoomType` (line 20) has no `resourceId`, so nothing returns which rooms passed. 5d would have to write that rule a second time, and two copies of it can drift (for example one checking the buffers and one not), which would let a booking take a room the free-time list never offered.

Suggested fix: When 5d is specced, name how it gets the free rooms: pull the room check out into one exported helper (for example `isRoomFree(room, date, spanStart, spanEnd)` in its own file) used by both `applyFreeTimesRules` and 5d, with `resourceId` on the room input. **Resolution:** Fixed 2026-10-02 in step 5d.3: `bookTime` picks its rooms with `isRoomFree` over `appointmentSpan`'s span (`book-time.ts`), the same rule and span as the free times; tested by "a room taken only during the buffer after is not chosen", which fails when the hold leaves the buffers out. Partly answered by step 5d.2, stays open: `backend/lib/scheduling/is-room-free.ts` is now the only copy of the rule and `applyFreeTimesRules` uses it (every 5c test unchanged and passing), and a room carrying its id still type-checks against `RoomScheduleType`, so 5d.3 can filter its own rooms. The booking, the second consumer this finding is about, does not exist yet; close it when `bookTime` picks its room with `isRoomFree` and a test shows it. See F-76 for the span the rule does not own. Not closed by the independent review of step 5d.3 (2026-10-02): the code does use `isRoomFree` (`book-time.ts:214`), but no saved test shows it. With `isRoomFree` mocked to always answer true, "a room taken only during the buffer after is not chosen" still passes (the database refuses Room 3 and decision 5 moves to Room 4), while a customer is booked into a room on standby; see F-79. Close it with F-79's test. Closed 2026-10-02 by independent review of step 5d.4 (7166169..ceb511c): `bookTime` still picks rooms with `isRoomFree` over `appointmentSpan`'s span (`book-time.ts:222-245`), and with `isRoomFree` mocked to always answer true, `book-time.test.ts` fails "a room on standby that date is not chosen for a customer, but the owner may use it" (1 failed, 19 passed).

5d/F-74 [P3] closed - "Any available" answers an empty week when every person's calendar is unreadable

File: `backend/lib/scheduling/find-free-times.ts:136` (test: `backend/routes/public-booking-links-routes.test.ts:391-393`) **Found:** 2026-10-02 by the final independent review of feature 5c (scope: `bf53ee6..bb2526d`; lenses: all) **Why it matters:** Decision 4 exists so "a broken calendar is never shown as an empty week": a picked person's unreadable calendar answers 503. With "any available" each unreadable person is left out (line 136-137 return `[]`), and nothing checks whether anyone was left to answer. When every candidate is left out, the route answers 200 with no start times, which a customer reads as "fully booked". The route test asserts exactly this for a one-person business (lines 391-393: 200, `startTimes: []`). For a one-person business (Primo's shape) or a Google-wide outage, "any available" is then the empty week decision 4 rules out for a pick, and the only trace is a console warning. The code follows the spec's letter; the spec does not say what happens when nobody is left. **Suggested fix:** Decide it in the spec (5d, or feature 9, where the widget chooses whether "any available" is its default): either answer 503 `unavailable` when every candidate was left out for an unreadable calendar, or keep 200 and say so in decision 4. Then make the route test say which. **Resolution:** Fixed in step 5d.5 (661b86e), per decision 9: `findFreeTimes` counts the calendars it read and the ones it could not; with "any available", none read and at least one failed throws `CalendarUnavailableError`, so the times route answers 503 `unavailable`. One readable calendar still answers 200. Tests: `find-free-times.test.ts` (no calendar

readable) and the route test now asserting 503. Closed 2026-10-02 by independent review of step 5d.5 (6906956..2663680): find-free-times.ts:157-159 throws CalendarUnavailableError only when no calendar was read and at least one failed, and the times route maps it to 503 `unavailable` (public-booking-links-routes.ts:118-124). With the throw removed, both "\"any available\" with no calendar readable is \"try again\" (find-free-times.test.ts) and \"a calendar that cannot be read is a 503\" (public-booking-links-routes.test.ts) fail; the existing \"left out of \"any available\" test (one readable, fully booked calendar beside an unreadable one) still answers 200 and would fail if the rule ignored the readable count. No new defect in the change.

5d/F-75 [P2] closed - Nothing in the schema or the plan makes "the same customer pressing Book twice gets one booking" hold when the two presses arrive together

File: packages/shared/db/booking-tables/booking-table.ts:64 (spec: blueprint/context/current-feature.md:66 and :184) **Found:** 2026-10-02 by independent review of step 5d.1 (scope: 7510d47..65250fa; lenses: all) **Why it matters:** For decision 7's lookup done one request after another, `booking_starts_at_index` (organizationId, startsAt) is enough: the join through `lead` to `contact.email` then touches a handful of rows. But nothing stops two presses that overlap in time. `booking` has no contact column and no unique key that could refuse a second booking for the same customer, service and start, and 5d.3 runs decision 7 "first, before the check", outside the transaction: a check-then-insert, which the spec's own notes rule out without a constraint behind it. Traced through the code that exists: A and B (same email, "any available", two free people) both find no booking and both pass the check. A inserts the contact; B's insert waits on `contact_organization_email_unique` (find-or-create-contact.ts:49) until A commits, then reads A's contact, writes a second lead and booking, is refused Ana by `commitment_no_overlap` and, by decision 5, holds the next person. One customer, two practitioners blocked at the same time. With a picked person B answers 409 `time_taken` to a customer who is in fact booked, where decision 7 and the contract promise 201 with the same booking. A double tap on a phone is the ordinary way this happens. 5d.3's Done when tests "booking twice" only one after the other. **Suggested fix:** Decide in 5d.3's plan, before it is built. Either (a) inside the transaction, right after `findOrCreateContact`, lock the contact row (`select ... for update`) and run decision 7's lookup again there, answering the existing booking if one appeared, so two presses for one email queue behind each other; or (b) while 0013 is unreleased, give `booking` the contact it is for and a partial unique index (organizationId, bookingLinkId, startsAt, contactId) where status is confirmed, so the database refuses the second (a contact without an email is always new, so owner-made bookings still book every time). Add a test: two identical "any available" requests at the same instant give one booking and both answer it. **Resolution:** Fixed 2026-10-02 with Frank's answer: neither suggested fix, because both refuse a parent booking two children for the same email, service and time. Instead a one-time requestKey per booking form (decision 7 rewritten): booking.requestKey with booking_request_key_unique on (organizationId, requestKey) where not null, migration 0014; the widget locks its Book button too (build plan item 9). The database test for the key is added and shown able to fail; 5d.3 handles a refused second copy and tests two copies at the same instant. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): the interleaving traced here (two copies both pass the check before

either commits) is now refused by `booking_request_key_unique` (0014) and answered with the booking that won (`book-time.ts:334-337`, `isSameRequest` guarding it); with that clash handling disabled in a scratch run, "two copies of one form at the same instant give one booking, and both answer it" fails. A different interleaving, where the second copy's check runs after the first has committed, still answers `time_taken` and is recorded as F-92; it is not a defect of this repair.

5d/F-76 [P3] closed - The room rule is one function now, but the span it checks (the appointment plus both buffers) is still worked out only inside `applyFreeTimesRules`

File: `backend/lib/scheduling/apply-free-times-rules.ts:79` (and `backend/lib/scheduling/is-room-free.ts:11`) **Found:** 2026-10-02 by independent review of step 5d.2 (scope: 7846a1d..230181c; lenses: all) **Why it matters:** `isRoomFree(room, date, spanStart, spanEnd)` takes the span ready-made, so the buffers are not part of the one rule. The only code that builds the span is lines 79-80 (`start - bufferBefore` , `start + duration + bufferAfter`). 5d.3 must build the same span twice more, for `isRoomFree` and for `holdTime` 's "buffers inside", and must load the rooms' busy time and standby dates again with their ids, because `findFreeTimes` builds its rooms without them (`find-free-times.ts:112-115`). F-64's own example of drift, "one checking the buffers and one not", is exactly the part `isRoomFree` does not guard: a booking whose span forgets `bufferAfterMinutes` would pass `isRoomFree` for a room the free-time list refused, and the hold would then also miss the buffer. Not observed: the booking code does not exist yet. **Suggested fix:** In 5d.3, export one small helper for the span (for example `appointmentSpan(start, service)` in its own file) used by `applyFreeTimesRules` , the room check and the hold, and have a 5d.3 test book a service with a buffer after next to a room taken only in that buffer. **Resolution:** Fixed 2026-10-02: `backend/lib/scheduling/appointment-span.ts`, `appointmentSpan(start, service)`, the one copy of the buffer before, the appointment and the buffer after; `applyFreeTimesRules` uses it, and 5d.3 must too (its `Done` when now names a room taken only during the buffer after). Tests added; with the buffer after dropped, 6 tests fail. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): `appointmentSpan` (`appointment-span.ts`) is the one span, used by `applyFreeTimesRules` (:80) and `bookTime` (:179) for both the room rule and the hold; with the buffer after dropped in a scratch run, 4 tests fail (`appointment-span.test.ts` and three in `book-time.test.ts`, including the room taken only during the buffer after). No new defect.

5d/F-77 [P3] closed - No saved test covers the deadlock retry inside a caller's transaction, which this step promises

File: `backend/lib/booking/writing-as-one-transaction.test.ts:67` (code: `backend/lib/scheduling/hold-time.ts:45`) **Found:** 2026-10-02 by independent review of step 5d.2 (scope: 7846a1d..230181c; lenses: tests) **Why it matters:** The spec (`current-feature.md:172-173`) and `hold-time.ts`'s header say a deadlock retry leaves the caller's transaction usable. The new tests only exercise the `23P01` path inside a transaction; the existing "many simultaneous holds" test uses the pool path only. The behaviour itself holds: a scratch probe ran 40 pairs of opposite-order holds (`[ana, room]` and `[room, ana]`), each inside its own `db.transaction` followed by a further query on that transaction, and `pg_stat_database.deadlocks` rose by 15 during the run; every pair gave one held and one taken,

nothing threw, all 80 outer transactions stayed usable. The same probe on the pool path saw 10 deadlocks, all resolved. So nothing is broken, but a later change (for example dropping the `per-try transaction()` on the pool path, or moving the retry outside the savepoint) could break the in-transaction path with every test still green. **Suggested fix:** Add the in-transaction twin of "many simultaneous holds" to `writing-as-one-transaction.test.ts`: each hold inside its own `db.transaction` that runs one more statement after it, asserting one held and one taken per pair and no throw. **Resolution:** Fixed 2026-10-02: `writing-as-one-transaction.test.ts` holds the same two people in opposite order inside two callers' transactions, ten times, each writing again after its hold; with the retry removed (`ATTEMPTS = 1`) it failed in 3 runs of 3. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): `writing-as-one-transaction.test.ts` holds two people in opposite order inside callers' transactions; with `ATTEMPTS = 1` in `hold-time.ts` in a scratch run it fails. No new defect.

5d/F-78 [P3] closed - The spec still says rooms carry their `resourceId`, which the step deliberately dropped

File: `blueprint/context/current-feature.md:176` **Found:** 2026-10-02 by independent review of step 5d.2 (scope: 7846a1d..230181c; lenses: quality) **Why it matters:** Step 5d.2's text says "rooms carry their `resourceId`", and the box is ticked, but `RoomScheduleType` (`is-room-free.ts:9`) has no id and `findFreeTimes` builds its rooms without one. The build log records the change ("the rooms did not have to carry their id"), but the commit only ticked the box, and the project rule is that a spec found wrong is corrected before the next step builds on it. 5d.3 is built from this file, which now describes a room shape that does not exist. **Suggested fix:** Amend 5d.2's bullet to what was built (`isRoomFree` takes any room with `busy` and `standbyDates`; the booking filters its own id-carrying rooms with it), and say in 5d.3 where those rooms and their ids are loaded. **Resolution:** Fixed 2026-10-02: the spec's 5d.2 now says `isRoomFree` takes any room with its taken time and standby dates and a caller keeps its own ids, with `appointmentSpan` as the one span. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): `current-feature.md` 5d.2 now says `isRoomFree` takes any room with its taken time and standby dates and a caller keeps its own ids, matching `is-room-free.ts` and `bookTime`'s id-carrying rooms (`book-time.ts:226-247`).

5d/F-79 [P3] closed - No saved test fails when `bookTime` stops filtering rooms with `isRoomFree`, so a room on standby could be booked for a customer unnoticed

File: `backend/lib/booking/book-time.ts:214` (test: `backend/lib/booking/book-time.test.ts:395`) **Found:** 2026-10-02 by independent review of step 5d.3 (scope: 4b04e58..285b930; lenses: tests) **Why it matters:** The room filter is load-bearing for one thing the database cannot refuse: a room on standby is hidden from customers but has no commitment, so only `isRoomFree` keeps it out of `choices`. The only room test, "a room taken only during the buffer after is not chosen", cannot fail when the filter goes: Room 3's time off makes `commitment_no_overlap` refuse the hold and decision 5 moves to Room 4, the same answer. A scratch Vitest run outside the repo with `is-room-free.ts` mocked to always answer true: that scenario still gave Room 4 (passed), and a message with Room 3 on standby that Monday booked the customer into Room 3 (with the real code, Room 4). Decision 11's other half, "the room must be free,

standby ignored" for the owner, is also untested (a scratch run shows the code books the owner into a standby room, as decided). The buffer before is never non-zero in any bookTime test, so the hold's span start is not pinned either. **Suggested fix:** Add saved tests: a room service with one room on standby that date, booked by a customer, takes the other room (and answers `time_taken` when every room is on standby); the same by the owner takes the standby room. Give one service a buffer before and assert the held commitment starts that much earlier. Then close F-64. **Resolution:** Fixed 2026-10-02: book-time.test.ts proves a room on standby that date is not chosen for a customer (fails with the room rule bypassed in bookTime) and that the owner may use it, and that a buffer before is held too (8:45 to 10:30 for a 9:00 facial with 15 before and 15 after). Closed 2026-10-02 by independent review of step 5d.4 (7166169..ceb511c): the standby and buffer-before tests are in book-time.test.ts (:418, :443) and pass at ceb511c; with isRoomFree bypassed by a scratch mock the standby test fails.

5d/F-80 [P3] closed - "Any available" with every calendar unreadable answers `time_taken`, which decision 12's message makes untrue

File: backend/lib/booking/book-time.ts:201-203 **Found:** 2026-10-02 by independent review of step 5d.3 (scope: 4b04e58..285b930; lenses: quality) **Why it matters:** A picked person whose calendar cannot be read answers `unavailable` (line 201), but with "any available" each unreadable person is dropped and, when nobody is left, line 203 answers `time_taken`. Scratch probe on a throwaway clinic with Ana and Mei both `needs_reconnect: {"booked":false,"reason":"time_taken"}`. 5d.5 turns that into "Sorry, that time was taken while you were booking", which is false: nothing was taken, Google could not be read. Decision 9 already rules this out for the times route (every calendar unreadable is 503, not an empty week); the booking does the opposite for the same state. A one-person business (Primo's shape) on "any available" during a Google outage that starts after the list loaded hits this exactly. The owner path does the same. **Suggested fix:** When no candidate is free and at least one was `unreadable`, answer `unavailable` (decision 9's rule, applied at booking time), and add the test beside "an unreadable picked calendar answers unavailable". **Resolution:** Fixed 2026-10-02: with nobody free and a calendar that could not be read, bookTime answers unavailable, never `time_taken`; tested with every practitioner's connection needing reconnection (fails if answered as taken). Closed 2026-10-02 by independent review of step 5d.4 (7166169..ceb511c): book-time.ts:220 answers unavailable when nobody is free and a calendar was unreadable; "any available with every calendar unreadable answers unavailable, not taken" (book-time.test.ts:457) passes.

5d/F-81 [P3] closed - A request key already booked answers that booking even when the service, time, person and customer differ

File: backend/lib/booking/book-time.ts:133-136 and :329-332 **Found:** 2026-10-02 by independent review of step 5d.3 (scope: 4b04e58..285b930; lenses: quality) **Why it matters:** `findBookedByRequestKey` matches on the key alone. Scratch probe: a facial at 9:00 for Jane booked with key K, then a second request with K for a massage at 11:00 for another name and email answered `{ booked: true, alreadyBooked: true }` with the facial at 9:00. Decision 7 covers a resend of the same form, but nothing says the resend is the same request. A widget that keeps one key while the customer,

already booked, changes the time or books a second child from the same open form would show a confirmation for something that was never booked (the 201 carries the real times, so only a careful widget notices). The usual idempotency-key convention (Stripe, the IETF Idempotency-Key draft) refuses a reused key whose request differs. No data leaks: the answer has no customer details. **Suggested fix:** Either compare `bookingLinkId`, `startsAt` and a picked `personId` with the stored booking and answer a refusal when they differ (a 409/422 at 5d.5), or write in decision 7 that a key lives only until its form books once and feature 9 must make a fresh key after every booking. Add the test either way. **Resolution:** Fixed 2026-10-02: a request key already booked answers that booking only for the same service and start (and the same person when one was picked); otherwise request_key_used, which 5d.5 turns into a 409 asking to reload. Tested with a different time, person and service (fails if any reuse returns the old booking). Closed 2026-10-02 by independent review of step 5d.4 (7166169..ceb511c): isSameRequest (book-time.ts:77-80) guards both the first lookup and the clash path (:149, :331); "a form already used for another booking is refused" (book-time.test.ts:479) passes.

5d/F-82 [P3] closed - An owner-made booking may start at any instant, the past included, and the spec does not say whether it should

File: backend/lib/booking/book-time.ts:164-198 (spec: blueprint/context/current-feature.md, decision 11) **Found:** 2026-10-02 by independent review of step 5d.3 (scope: 4b04e58..285b930; lenses: quality) **Why it matters:** Decision 11 lifts notice and the horizon for the owner, and the owner path checks only busy time, so nothing bounds the start. Scratch probe: `source: "manual"` for Mei at 2020-01-01 09:00 Edmonton answered `booked: true`. Recording a walk-in after the fact may be wanted, but 5d.4 will write a Google event for it and features 6 and 8 will send a confirmation and a reminder for a booking years in the past, and a typo'd year (2062) would sit in a worker's calendar. The decision is Frank's; the code made it silently. **Suggested fix:** Decide in decision 11 (with the owner's route, feature 11 or 12b, at the latest): either the owner may book the past (and later features skip messages for it), or bookTime refuses a start before `now` (and, perhaps, beyond some outer limit). Add the test for whichever. **Resolution:** Fixed 2026-10-02 with Frank's answer (option B, decision 13): an owner-made booking may start any time from the start of today in the business's zone; an earlier day answers `in_the_past`. Tested with a walk-in begun at 10:00 entered at 10:10 (booked), yesterday and 2020 (refused), and just after local midnight (booked); the test fails both with no limit and with "only from now on". Closed 2026-10-02 by independent review of step 5d.4 (7166169..ceb511c): book-time.ts:175 answers `in_the_past` for an owner start on an earlier local day; "an owner-made booking may start earlier today, never on an earlier day" (book-time.test.ts:553) passes.

5d/F-83 [P3] closed - The try order is built by calling chooseAnyAvailable in loops with a stand-in person, and the choice type is declared twice

File: backend/lib/booking/book-time.ts:244-263 (and hold-first-free-choice.ts:8, choose-any-available.ts:7) **Found:** 2026-10-02 by independent review of step 5d.3 (scope: 4b04e58..285b930; lenses: quality) **Why it matters:** To sort people and rooms by decision 2's order without a second copy of the rule, bookTime picks one with `chooseAnyAvailable`, removes it and asks again, and for rooms

passes a made-up person (`{ resourceId: people[0], name: "", bookingsThatDay: 0 }`) so the function will answer a room. It is correct but takes a careful read to see that it is only a sort. `BookingChoiceType` (`{ personId; placeId: string | null }`) is the same shape as the existing `AnyAvailableChoiceType` . Comments also cite feature and step numbers ("5c's free times", "as 5c reads them", `book-time.ts:4, :37, :164, :200`), which `coding-standards.md` lists as history (older files do the same). **Suggested fix:** Export the ordering from `choose-any-available.ts` (for example `orderAnyAvailable(people, rooms)` answering every choice in order, with `chooseAnyAvailable` its first element) and use it in `bookTime`; reuse `AnyAvailableChoiceType` in `holdFirstFreeChoice`; say "the free times" instead of "5c's". **Resolution:** Fixed 2026-10-02: the order lives in one place, `backend/lib/scheduling/order-any-available.ts` (`orderAnyAvailable`, with its own tests); `chooseAnyAvailable` is its first choice and `bookTime` uses the whole order, so no made-up person and no loop. `BookingChoiceType` is gone in favour of `AnyAvailableChoiceType`. Comments no longer name 5c. Closed 2026-10-02 by independent review of step 5d.4 (7166169..ceb511c): `order-any-available.ts` holds the order and `AnyAvailableChoiceType`, `chooseAnyAvailable` returns its first element, `hold-first-free-choice.ts` uses `AnyAvailableChoiceType`, `BookingChoiceType` is gone, and no comment in `backend/lib/booking` names 5c.

5d/F-84 [P3] closed - Writing a booking's event twice makes a second Google event and forgets the first, and a cancelled booking is written too

File: `backend/lib/calendar/write-booking-event.ts:64-76` (and `google-calendar-provider.ts:64`) **Found:** 2026-10-02 by independent review of step 5d.4 (scope: 7166169..ceb511c; lenses: all) **Why it matters:** `writeBookingEvent` checks neither `calendarEventId` nor `status` , and Google's `events.insert` is not idempotent. Scratch probe on a throwaway clinic with Google faked: calling it again for a booking already holding `evt-1` posted a second event and replaced the id with `evt-3` , so the first event stays in the worker's calendar with nothing pointing at it, and feature 7's cancel can never remove it. A booking set to `cancelled` was still written (`evt-4`). Today `bookTime` calls it once per new booking, so nothing double-writes yet. Feature 8's retry (decision 6) will, and the ten-second limit makes the bad case ordinary: Google can create the event and the answer time out (or the `calendarEventId` update fail), leaving the id empty for a booking whose event exists, which the retry then writes again. A worker could turn up for a cancelled job because its twin was never removed. **Suggested fix:** Send a client-chosen event `id` derived from the booking id (Google accepts base32hex, a-v and 0-9, 5 to 1024 characters; the UUID without its hyphens qualifies), so a second insert answers 409 and is treated as already written; and have `writeBookingEvent` skip a booking that is cancelled or already has `calendarEventId` (or settle both in feature 8's spec). Add a test that writes twice and sees one event. **Resolution:** Fixed 2026-10-02: the event's id is the booking's without dashes (Google's a-v0-9 rule), so Google keeps one event; a 409 for that id is taken as the event already made; a booking already written answers its saved id without asking Google, and a cancelled one is not written. Tests: writing again makes no second call, a 409 saves the id, a cancelled booking gets no event; each fails with its guard removed. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): `write-booking-event.ts:60-66` skips a cancelled or already written booking and sends the dashless booking id; `google-calendar-provider.ts` treats 409 as the

event already made. In scratch runs each guard removed (the saved-id return, the status check, the 409 branch) fails its own test in write-booking-event.test.ts. No new defect.

5d/F-85 [P3] closed - A returning customer's phone typed on this booking never reaches the event

File: backend/lib/calendar/write-booking-event.ts:24-26 and :61 (cause: backend/lib/crm/find-or-create-contact.ts:70) **Found:** 2026-10-02 by independent review of step 5d.4 (scope: 7166169..ceb511c; lenses: quality) **Why it matters:** Decision 14 puts "Phone: ..." in the description when given. The event reads the phone from `contact`, and a known email keeps "the name and phone first given". Scratch probe: Jane booked one Monday with email only, then the next Monday with the same email and phone 403 555 0199; the second event's description was exactly `Email: ...`, no phone line, and the phone is stored nowhere (the lead keeps only `details`). A painter sent to a returning customer gets no phone, or the old one if it changed; the title likewise carries the first name ever given. **Suggested fix:** Frank's call. Either fill a known contact's empty phone (or replace it) in `findOrCreateContact`, or keep this booking's phone on the lead or booking and use it for the event; or write in decision 14 that the contact's stored details are what the event shows. Add the test either way. **Resolution:** Fixed 2026-10-02 with Frank's answer (option A, decision 15): `lead.phone` (migration 0015) keeps the phone given with each request and the event shows it; `findOrCreateContact` fills a known contact's phone only when it had none. Tests: a returning customer's new phone reaches this booking's event (fails when the event reads the contact's phone), and a known contact is filled once, never replaced (fails when the empty-phone guard is removed). Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): `lead.phone` (0015) is written by `bookTime` (book-time.ts:282) and preferred by the event (write-booking-event.ts:72); `findOrCreateContact` fills a phone only where it is null. Reading the contact's phone first fails "a returning customer's new phone reaches this booking's event", and dropping the null guard fails two find-or-create-contact tests (scratch runs). No new defect.

5d/F-86 [P3] closed - Two Done-when items cannot fail: the refresh test passes when writeBookingEvent never refreshes, and "each only when given" is never tested

File: backend/lib/calendar/write-booking-event.test.ts:249 and :160 **Found:** 2026-10-02 by independent review of step 5d.4 (scope: 7166169..ceb511c; lenses: tests) **Why it matters:** In "an expired token is refreshed through the one shared helper", `bookTime`'s own check reads the busy times first, and `getBusyTimes` renews the 30-second token (call order in a probe: token, freeBusy, events), so `writeBookingEvent` finds a fresh token already stored. A scratch Vitest run with `writeBookingEvent`'s `getFreshAccessToken` replaced by a plain read of the stored token (no renewal at all) passed every assertion of that test (one token call, `Bearer ya29.fresh-access`). The content test has phone, email and details all present, so removing the "only when given" filtering (write-booking-event.ts:61-66) fails nothing, though that rule is the whole of decision 14's description. Also, reconnection and the timeout are tested by calling `writeBookingEvent` directly, so "keeps the booking and logs one line" is shown through `bookTime` only for the Google error. **Suggested fix:** Test the renewal by calling `writeBookingEvent` directly for a booking made before connecting, with under a minute left on the token, asserting the token call and

the Bearer sent to the event; add content cases with no phone and no details (`Email: ...` only) and with phone only. **Resolution:** Fixed 2026-10-02: the refresh test now books before connecting and writes the event directly, so only the event write can renew the key (fails when the expired key is used anyway); a new test books with no phone and no note and checks the description is the email line alone (fails when missing lines are kept). Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): with renewal skipped in `get-fresh-access-token.ts`, "an expired token is refreshed through the one shared helper" fails; with every line kept regardless of what was given, "a phone, an email or a note left out leaves its line out" fails (scratch runs).

5d/F-87 [P3] closed - The token helper's comment describes a condition no longer written that way, and two header comments were left unwrapped

File: `backend/lib/calendar/get-fresh-access-token.ts:65` (also `backend/lib/booking/book-time.ts:4`, `backend/lib/calendar/calendar-provider.ts:2`) **Found:** 2026-10-02 by independent review of step 5d.4 (scope: 7166169..ceb511c; lenses: quality) **Why it matters:** "Written as 'not comfortably valid', so an unreadable date refreshes too" explained the old `!(expiresAt - Date.now() > margin)`; the condition is now the opposite test with an early return. Behaviour is unchanged (NaN compares false, so an unreadable date still falls through to the renewal; compared line by line with `get-busy-times.ts` at 7166169, the race rules, the `needs_reconnect` marking and both `continue` s are identical), but the comment now names a form that is not there, on the one subtlety a reader must not "simplify" away. `book-time.ts:4` (149 characters) and `calendar-provider.ts:2` (129) had words inserted into wrapped headers without rewrapping (Prettier leaves comments alone), and the new import in `book-time.ts:17` sits out of order. **Suggested fix:** Say what the line does now ("only a key comfortably valid is used as it is; an unreadable date fails this test and is renewed"), rewrap the two headers, sort the import. **Resolution:** Fixed 2026-10-02: the comment in `get-fresh-access-token.ts` now matches its condition; the two long header comments are rewrapped under 100 characters; `book-time.ts`'s calendar imports are in order. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): `get-fresh-access-token.ts:156-157` now describes the condition as written (a comfortably valid key is used, anything else, an unreadable date too, is renewed); the two headers are under 100 characters and the imports are in order.

5d/F-88 [P3] closed - The POST preflight test passes even when the browser would be refused the JSON header every booking sends

File: `backend/routes/public-bookings-routes.test.ts:322` **Found:** 2026-10-02 by independent review of step 5d.5 (scope: 6906956..2663680; lenses: tests) **Why it matters:** The widget posts `Content-Type: application/json`, which makes the browser send a preflight and book only if the answer allows that header. Today Hono's cors reflects the requested headers because `publicCorsMiddleware` sets no `allowHeaders`, so it works. The test sends `Access-Control-Request-Headers: content-type` but never checks `Access-Control-Allow-Headers`: with `allowHeaders: ["x-nothing"]` added to `public-cors-middleware.ts` in a scratch run, all 17 tests in the file still passed, while every real browser booking would be blocked. The Done when's "a preflight allows POST" is the one browser-facing promise of this

step. **Suggested fix:** Assert that `Access-Control-Allow-Headers` includes `content-type` (case-insensitive) in the same test. **Resolution:** Fixed in 5d.5's review fixes (5ac0594): the preflight test now also asserts `Access-Control-Allow-Headers` includes `content-type`. Proved able to fail: with `allowHeaders ["x-nothing"]` in `publicCorsMiddleware` the test fails. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): the preflight test asserts `Access-Control-Allow-Headers` contains `content-type`; with `allowHeaders ["x-nothing"]` in `publicCorsMiddleware` in a scratch run it fails.

5d/F-89 [P3] closed - `CreateBookingInputType` is exported and used nowhere; the typed client already carries the body type

File: `packages/shared/zod-validation/booking-links-validation-schemas/create-booking-validation-schema.ts:30` **Found:** 2026-10-02 by independent review of step 5d.5 (scope: 6906956..2663680; lenses: quality) **Why it matters:** No file in backend or frontend imports it. The widget (feature 9) calls the route through `hc<AppType>`, which takes the body type from `validator("json")`, so this second declaration of the same shape is not needed by the planned consumer either. The other booking-link schemas export no input type. "What breaks if we do not add this?" has no answer yet. **Suggested fix:** Remove the export; add it back if a consumer appears that the typed client does not serve. **Resolution:** Fixed in 5d.5's review fixes (5ac0594): the unused `CreateBookingInputType` export is removed; the typed client takes the body type from the validator. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): `create-booking-validation-schema.ts` exports only the schema; nothing imports a booking input type.

5d/F-90 [P3] closed - The public CORS header comment had a sentence inserted without rewrapping (139 characters)

File: `backend/middleware/public-middleware/public-cors-middleware.ts:2` **Found:** 2026-10-02 by independent review of step 5d.5 (scope: 6906956..2663680; lenses: quality) **Why it matters:** Prettier leaves comments alone, so `format:check` passes, but the header now runs to 139 characters against the 100 every other line keeps. Same pattern as F-87 (words added to a wrapped header in step 5d.4), so it is drift rather than a one-off. **Suggested fix:** Rewrap the three header lines under 100 characters. **Resolution:** Fixed in 5d.5's review fixes (5ac0594): the header comment is rewrapped under 100 characters. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): the header comment of `public-cors-middleware.ts` is three lines under 100 characters.

5d/F-91 [P3] closed - The step's Done when says the route tests run on clinic-dev; they run on a clinic of their own

File: `blueprint/context/current-feature.md:309` **Found:** 2026-10-02 by independent review of step 5d.5 (scope: 6906956..2663680; lenses: tests) **Why it matters:** `public-bookings-routes.test.ts` builds and removes its own clinic, so a failed run never leaves bookings in the seeded one (its header says why), and uses clinic-dev only for "another business's service". That is the better choice and matches the spec's Testing section, but the Done when still names clinic-dev, so the spec and the tests disagree on what was

proved and where. **Suggested fix:** Change the Done when to "route tests on a clinic of their own (and clinic-dev for another business's service)". **Resolution:** Fixed in 5d.5's review fixes (5ac0594): the Done when now says the route tests run on a clinic of their own, removed after, and why. Closed 2026-10-02 by the final independent review of feature 5d (12a21d6..aa237da): current-feature.md 5d.5's Done when names a clinic of their own, removed after, matching public-bookings-routes.test.ts.

5d/F-92 [P2] closed - A resent form whose check runs after its first copy committed answers time_taken, though the customer is booked

File: backend/lib/booking/book-time.ts:220-223 (also :248 and :332; the key is read only at :150-154)

Found: 2026-10-02 by the final independent review of feature 5d (scope: 12a21d6..aa237da; lenses: quality, security, performance, tests) **Why it matters:** Decision 7 promises that the same form sent twice gets one booking and both copies answer it. bookTime reads the request key once, before the check, and consults it again only when its own booking insert clashes on `booking_request_key_unique`. When the second copy reads the key before the first commits but runs its check after (a press repeated after a slow or lost answer, arriving while the first is still being written), the check sees the first copy's own commitment and answers `time_taken` (picked person, line 223), `unavailable` (line 220 or 223) or `time_taken` for the rooms (line 248), without ever reaching the insert. Shown in a scratch Vitest run (since removed) that held the second copy's check until the first copy had booked: the first answered booked, the second `{ booked: false, reason: "time_taken" }` for the same key, service, person and start. The customer reads "Sorry, that time was taken while you were booking" for a booking that exists; picking another time on the same form then gets `request_key_used`, and after a reload they book a second time, so the business holds two bookings for one job. The saved test for two copies at the same instant uses "any available" with two free people, so the second copy always finds someone free and reaches the insert; a picked person, a one-person business (Primo's shape) or a room service is never tested. **Suggested fix:** Before answering `time_taken` or `unavailable` (lines 220, 223, 248 and the `EveryChoiceTakenError` branch at 332) when a `requestKey` was sent, read `findBookedByRequestKey` again and answer that booking (or `request_key_used`) through `isSameRequest`, as the clash path already does. Add a test with a picked person whose check runs after the first copy commits. **Resolution:** Fixed after the final review: bookTime asks the form's key again before every refusal (the picked person unreadable or busy, nobody free, no free room, every choice taken), through one helper used by the first lookup and the clash path too, so a copy whose check ran after its first copy was saved answers that booking. Test: book-time-resent-while-saving.test.ts holds the second copy's check until the first has booked, with one painter; it fails without the fix (`time_taken`) and passes with it. Closed 2026-10-02 by the second final independent review of feature 5d (12a21d6..3cec4ae): every refusal that the first copy's own booking can cause (book-time.ts:230 picked person unreadable, :234 nobody free, :260 no free room, :338 every choice taken) now answers through `refuseUnlessBooked`, which rereads the key and answers that booking or `request_key_used` through `isSameRequest`; a booking and its commitments become visible in one commit, so a copy that sees the time taken also sees the key. With `refuseUnlessBooked` reduced to the plain refusal (temporarily,

reverted), book-time-resent-while-saving.test.ts failed with `{ booked: false, reason: "time_taken" }`; restored, it passes. The repair adds one key read on refusals only and no new path.

5d/F-93 [P3] closed - The customer's answer waits for Google's event write, up to 10 seconds (20 with a key renewal), after the booking is already saved

File: backend/lib/booking/book-time.ts:319-325 (limits: backend/lib/calendar/google-calendar-provider.ts:15, backend/lib/calendar/google-oauth-client.ts:13) **Found:** 2026-10-02 by the final independent review of feature 5d (scope: 12a21d6..aa237da; lenses: quality, security, performance, tests) **Why it matters:** bookTime awaits `writeBookingEvent` before returning, and the public route answers only then. The answer does not depend on the event (decision 6 keeps the booking whatever Google does), but when Google is slow, which is exactly the case decision 6 is about, a customer whose booking was saved in milliseconds watches the Book button spin for up to 10 seconds, or 20 when the key is renewed first. A widget or browser that gives up sooner shows a failure for a booking that exists. The coding standards say long running work does not belong in a request handler; the spec only says the write happens after the commit, so the wait is a side effect nobody decided. **Suggested fix:** Frank's call. Either answer first and let the write finish on its own (not awaited, its catch and warning line kept) until feature 8's job runner owns it, or write in decision 6 that the customer's answer waits for Google until then. **Resolution:** Frank's answer, option A (2026-10-02, written into decision 6): bookTime starts the event write once the booking is saved and does not await it (`bookingEventWrites.start` in backend/lib/booking/booking-event-writes.ts, which keeps the catch and the one warning line and lets tests await the writes still running). Test: "the customer's answer does not wait for Google" holds Google's answer and gets the booking first; with the wait put back it times out. Closed 2026-10-02 by the second final independent review of feature 5d (12a21d6..3cec4ae): book-time.ts:331 calls `bookingEventWrites.start` after the transaction and returns without awaiting it; booking-event-writes.ts attaches `.catch` (one warning line, booking id and `safeErrorReason` only) before anything can reject, so a Google failure can never become an unhandled rejection, and `.finally` drops the write from the running set. With `await bookingEventWrites.settled()` added after the start (temporarily, reverted), "the customer's answer does not wait for Google" failed on its timeout; restored, it passes. The repair introduced no new defect in these two files (a test-cleanup gap in the same test is its own entry, F-95).

Independent review

Status: passed **Target commit:** 3cec4aebb33ebf326d2df53748e4f269334e2bb9 **Base commit:** 12a21d6b803c700faa23a29df7d974039481a115 **Base ref:** main **Spec hash:** 3674ff04151a7e2f1f20a8bc529920411de54f434972aa52d10f7604f3d13708 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-03T00:16:01Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh

subagent **Actual execution:** automatic **Reviewed at:** 2026-10-03T00:23:01Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`12a21d6b803c700faa23a29df7d974039481a115..3cec4aebb33ebf326d2df53748e4f269334e2bb9` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain --untracked-files=all` : pass (target, base and spec hash match; only review.md differed)
- `npm run test --workspace=@scheduleads-app/shared` : pass (11 files, 90 tests)
- `npm run test --workspace=backend` : pass (37 files, 358 tests, against the local scheduleads_dev)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass (typechecks the typed client against the backend's route types)
- `npm run format:check` : pass
- `npm run db:generate --workspace=@scheduleads-app/shared` : pass ("No schema changes, nothing to migrate"; tree unchanged after)
- `npx vitest run lib/booking/book-time-resent-while-saving.test.ts` with `refuseUnlessBooked` temporarily reduced to the plain refusal: fail as expected (time_taken), code reverted
- `npx vitest run lib/calendar/write-booking-event.test.ts -t "does not wait"` with the event write temporarily awaited: fail as expected (timeout), code reverted

Evidence

- Whole delta read (60 files): migrations 0013 to 0015 and the lead, booking and commitment tables; the executor argument and savepoint in holdTime, releaseTime, findOrCreateContact, recordActivity; isRoomFree, appointmentSpan, overlapsAny, orderAnyAvailable; bookTime, holdFirstFreeChoice, bookingEventWrites, writeBookingEvent, getFreshAccessToken, the Google createEvent; POST /public/:slug/bookings, the CORS change, the times route's 503; every new or changed test.
- Tenant scoping: every query in bookTime, findBookedByRequestKey, namesOf, writeBookingEvent and the route's name reads filters on the organizationId taken from the slug; foreign keys carry

(organizationId, id) pairs, proved by lead-and-booking-rules.test.ts.

- Customer details: the 201 body holds ids, times, zone, service and person only (route test checks name, email, phone, address, words and organizationId are absent); log lines carry resource or booking ids and safeErrorReason, which reduces database errors to their Postgres code; the activity payload holds ids and the start only.
- Concurrency: a booking and its commitments become visible in one commit, so a copy that sees the time taken also sees the form's key (refuseUnlessBooked at book-time.ts:230, :234, :260, :338); copies racing to the insert block on booking_request_key_unique and the loser answers the winner; holds run in savepoints so a refusal or deadlock retry keeps the transaction usable.
- Unawaited work: the only unawaited promise is bookingEventWrites.start, whose catch is attached before it can reject; no unhandled rejection or process crash path found. Google calls have a 10-second limit; the token refresh shares the race-safe helper.
- Decisions 1 to 16 checked against the code; contracts (body schema, status table, bookTime result, event shape and id) match.
- Ledger: F-92 and F-93 closed with evidence; F-94 and F-95 added (P3).

Findings

- F-94 [P3] open: chooseAnyAvailable has no caller outside its own test, while the spec still says the booking's order comes from it
- F-95 [P3] open: when the no-wait test fails, its cleanup hangs on the held Google answer and leaves its business in the dev database
- F-92 [P2] closed, F-93 [P3] closed

Remaining risk

- Check was not required and not run: no browser or running-server evidence; the route was exercised through app.request against the local database.
- Google is faked in every test; the real events.insert (including its 409 on a reused id) is unproven against Google itself.
- A background event write lost to a server restart waits for feature 8's retry (decision 6, accepted).
- No rate limit on the public POST yet (out of scope, planned for feature 9).
- This review's deliberate failing run left one throwaway business (`test-event-no-wait-<tag>`) in scheduleads_dev; another from an earlier run is also there. Harmless, removed by a rebuild of the dev database (F-95).